COMPTE RENDU DE TRAVAUX PRATIQUES

TP 1 – Python

Partie 1

Conversions, conditions, boucles, fonctions, listes et structures de données pour l'IA

Réalisé par : Ayman Joundi

Filière / groupe : AIDC

Encadrant(e) : Lhoussaine Daoudi

Année universitaire : 2026–2027

Table des matières

1 Introduction **2**

2 Environnement et méthode **2**

3 Exercice 1 – Calcul d'une moyenne pondérée **2**
3.1 Objectif et méthode 2
3.2 Exemple de vérification 2

4 Exercice 2 – Analyse d'une équation **3**
4.1 Objectif et méthode 3
4.2 Exemple de vérification 3

5 Exercice 3 – Validation d'un entier impair **4**
5.1 Objectif et méthode 4
5.2 Exemple de vérification 4

6 Exercice 4 – Fonctions, map et zip **5**
6.1 Objectif et méthode 5

7 Exercice 5 – Notes triées et médiane **6**
7.1 Objectif et méthode 6
7.2 Exemple de vérification 6

8 Exercice 6 – Configuration d'un système multi-agents **7**
8.1 Objectif et méthode 7

9 Bilan et conclusion **7**

1. Introduction

Ce travail pratique permet de maîtriser les fondements de la programmation Python à travers six exercices progressifs. Les solutions ont été développées dans un notebook Jupyter compatible avec Google Colab. Le présent rapport explique les choix algorithmiques et prévoit une capture d'écran pour chaque exécution. Les captures devront être ajoutées après exécution réelle du notebook.

2. Environnement et méthode

1. Ouvrir Google Colab.
2. Importer le fichier `TP1_Python_Partie1_Corrige_Colab.ipynb`.
3. Exécuter les cellules successivement et répondre aux demandes de saisie.
4. Prendre des captures où les valeurs entrées et les résultats affichés sont visibles.
5. Enregistrer les captures sous les noms indiqués dans ce rapport, puis compiler le fichier LaTeX avec pdfLaTeX dans Overleaf.

3. Exercice 1 – Calcul d'une moyenne pondérée

3.1. Objectif et méthode

Recueillir l'identifiant de l'étudiant, deux notes décimales et leurs coefficients entiers. L'identifiant reste une chaîne, les notes utilisent `float` et les coefficients `int`. La formule est :

$$M = \frac{N_{TP}C_{TP} + N_E C_E}{C_{TP} + C_E}.$$

Avant la division, il faut vérifier que la somme des coefficients n'est pas nulle. Le résultat est converti en chaîne avec `str()` pour composer la phrase demandée.

3.2. Exemple de vérification

Pour $N_{TP} = 16$, $C_{TP} = 2$, $N_E = 15$, $C_E = 3$:

$$M = \frac{16 \times 2 + 15 \times 3}{2 + 3} = 15{,}4.$$

```
# Exercice 1 : Moyenne pondérée
identifiant = input("Identifiant de l'étudiant : ")
note_projet = float(input("Note du projet /20 : "))
note_ecrit = float(input("Note de l'examen /20 : "))
coeff_projet = int(input("Coefficient du projet : "))
coeff_ecrit = int(input("Coefficient de l'examen : "))

if coeff_projet + coeff_ecrit == 0:
    print("Erreur : la somme des coefficients ne peut pas être nulle.")
else:
    moyenne = (note_projet * coeff_projet + note_ecrit * coeff_ecrit) / (coeff_projet + coeff_ecrit)
    print("Pour l'étudiant " + identifiant + ", la moyenne finale du module est de " + str(round(moyenne, 2)) + " / 20.")
```

```
Identifiant de l'étudiant : CNE12345
Note du projet /20 : 16
Note de l'examen /20 : 15
Coefficient du projet : 2
Coefficient de l'examen : 3
Pour l'étudiant CNE12345, la moyenne finale du module est de 15.4 / 20.
```

FIGURE 1 – Exercice 1 : saisies et moyenne pondérée dans Colab

4. Exercice 2 – Analyse d'une équation

4.1. Objectif et méthode

Étudier $ax^2+bx+c=0$ en respectant deux structures conditionnelles successives. Le premier bloc teste si $a=0$; un test imbriqué permet de distinguer l'équation du premier degré et les cas dégénérés, et renseigne le booléen `est_second_degre`. Un deuxième `if` indépendant calcule le discriminant uniquement si l'équation est bien du second degré.

$$\Delta = b^2 - 4ac.$$

Si $\Delta > 0$, il existe deux solutions réelles distinctes :

$$x_{1,2} = \frac{-b \pm \sqrt{\Delta}}{2a}.$$

Si $\Delta = 0$, il existe une racine double $x = -b/(2a)$. Si $\Delta < 0$, il n'existe aucune racine réelle. Pour $a = 0$ et $b \neq 0$, l'unique racine est $x = -c/b$; si $a = b = 0$, le cas dépend de c.

4.2. Exemple de vérification

Avec $a=1$, $b=-3$, $c=2$, on obtient $\Delta = 1$, puis $x_1 = 1$ et $x_2 = 2$.

```python
# Exercice 2 : Analyse d'une équation ax² + bx + c = 0
a = float(input("Coefficient a : "))
b = float(input("Coefficient b : "))
c = float(input("Coefficient c : "))

if a == 0:
    if b == 0:
        if c == 0:
            print("Équation dégénérée : une infinité de solutions.")
        else:
            print("Équation dégénérée : aucune solution.")
    else:
        print("Équation du premier degré : x =", -c / b)
    est_second_degre = False
else:
    est_second_degre = True

# Structure conditionnelle indépendante de la précédente
if est_second_degre:
    delta = b**2 - 4*a*c
    print("Discriminant :", delta)
    if delta > 0:
        x1 = (-b - delta**0.5) / (2*a)
        x2 = (-b + delta**0.5) / (2*a)
        print("Deux solutions réelles :", x1, "et", x2)
    elif delta == 0:
        x = -b / (2*a)
        print("Une solution réelle double :", x)
    else:
        print("Aucune solution réelle.")
```

```
Coefficient a : 1
Coefficient b : -3
Coefficient c : 2
Discriminant : 1.0
Deux solutions réelles : 1.0 et 2.0
```

FIGURE 2 – Exercice 2 : discriminant et solutions

5. Exercice 3 – Validation d'un entier impair

5.1. Objectif et méthode

Une boucle `while` répète la saisie jusqu'à l'obtention d'un entier impair valide. Une gestion des erreurs de conversion rejette les chaînes non entières et les décimaux. La solution exige aussi un entier positif pour que l'affichage de 0 jusqu'au nombre saisi soit cohérent. Une boucle `for` avec `range(0, nombre+1)` affiche la borne supérieure incluse.

5.2. Exemple de vérification

En saisissant successivement `4`, `3.5`, puis `5`, les deux premières valeurs sont refusées et la dernière permet d'afficher 0, 1, 2, 3, 4, 5.

```python
# Exercice 3 : Validation et affichage des nombres
while True:
    saisie = input("Veuillez saisir un nombre entier impair positif : ")
    try:
        nombre = int(saisie)
    except ValueError:
        try:
            float(saisie)
            print("Erreur : vous avez saisi un nombre décimal ou non entier.")
        except ValueError:
            print("Erreur : saisissez un nombre entier valide.")
        continue

    if nombre < 0:
        print("Erreur : veuillez saisir un nombre positif pour afficher de 0 à ce nombre.")
    elif nombre % 2 == 0:
        print("Erreur :", nombre, "est un nombre pair.")
    else:
        break

print("Saisie acceptée ! Voici les nombres de 0 à", nombre, ":")
for i in range(0, nombre + 1):
    print(i)
```

```
Veuillez saisir un nombre entier impair positif : 4
Erreur : 4 est un nombre pair.
Veuillez saisir un nombre entier impair positif : 3.5
Erreur : vous avez saisi un nombre décimal ou non entier.
Veuillez saisir un nombre entier impair positif : 5
Saisie acceptée ! Voici les nombres de 0 à 5 :
0
1
2
3
4
5
```

FIGURE 3 – Exercice 3 : refus des saisies invalides puis affichage de la séquence

6. Exercice 4 – Fonctions, map et zip

6.1. Objectif et méthode

La fonction `ajouter_bonus(note, bonus=2)` renvoie la somme de la note et du bonus. Avec `x=10`, son appel donne 12, tout en conservant `x=10` : les entiers Python sont immuables. La fonction `map()` applique le bonus aux notes `[12.0, 15.5]` et produit `[14.0, 17.5]`. Enfin, `zip()` associe les noms `Ali` et `Sara` à ces notes, dans le même ordre.

```python
# Exercice 4 : Fonctions, immuabilité, map et zip
def ajouter_bonus(note, bonus=2):
    return note + bonus

x = 10
resultat = ajouter_bonus(x)
print("Note avec bonus :", resultat)
print("Valeur initiale de x (inchangée) :", x)

notes = [12.0, 15.5]
notes_bonifiees = list(map(ajouter_bonus, notes))
print("Notes après bonus :", notes_bonifiees)

etudiants = ["Ali", "Sara"]
for etudiant, note in zip(etudiants, notes_bonifiees):
    print(etudiant, ":", note)
```

```
Note avec bonus : 12
Valeur initiale de x (inchangée) : 10
Notes après bonus : [14.0, 17.5]
Ali : 14.0
Sara : 17.5
```

FIGURE 4 – Exercice 4 : bonus, immuabilité, map et zip

7. Exercice 5 – Notes triées et médiane

7.1. Objectif et méthode

L'utilisateur indique le nombre de notes, puis les saisit dans l'ordre croissant. Une boucle `while` refuse toute note inférieure à la dernière saisie. Dans cette solution, les notes sont aussi contrôlées sur l'intervalle de 0 à 20. Comme la liste est déjà triée, aucun tri supplémentaire n'est nécessaire.

$$\mathrm{Med}(n) = \begin{cases} L[n//2], & n \text{ impair}, \\ \dfrac{L[n//2-1] + L[n//2]}{2}, & n \text{ pair}. \end{cases}$$

7.2. Exemple de vérification

Avec les notes $10, 12, 15, 18$, la médiane est $(12+15)/2 = 13{,}5$. Avec $10, 12, 15$, la médiane est 12.

```python
# Exercice 5 : Saisie croissante et médiane
while True:
    try:
        n = int(input("Combien de notes souhaitez-vous saisir ? "))
        if n > 0:
            break
        print("Erreur : le nombre de notes doit être positif.")
    except ValueError:
        print("Erreur : saisissez un nombre entier.")

notes = []
for i in range(n):
    while True:
        try:
            note = float(input(f"Note {i + 1}/{n} (ordre croissant) : "))
            if note < 0 or note > 20:
                print("Erreur : une note doit être entre 0 et 20.")
            elif notes and note < notes[-1]:
                print("Erreur : la note doit être supérieure ou égale à", notes[-1])
            else:
                notes.append(note)
                break
        except ValueError:
            print("Erreur : saisissez un nombre valide.")

milieu = n // 2
if n % 2 == 1:
    mediane = notes[milieu]
else:
    mediane = (notes[milieu - 1] + notes[milieu]) / 2

print("Notes triées :", notes)
print("Nombre de notes :", n, "(" + ("pair" if n % 2 == 0 else "impair") + ")")
print("Médiane :", mediane)
```

```
Combien de notes souhaitez-vous saisir ? 4
Note 1/4 (ordre croissant) : 10
Note 2/4 (ordre croissant) : 12
Note 3/4 (ordre croissant) : 15
Note 4/4 (ordre croissant) : 18
Notes triées : [10.0, 12.0, 15.0, 18.0]
Nombre de notes : 4 (pair)
Médiane : 13.5
```

FIGURE 5 – Exercice 5 : saisie des notes et calcul de la médiane

8. Exercice 6 – Configuration d'un système multi-agents

8.1. Objectif et méthode

La structure `config_ia` est un dictionnaire. Sa clé `agent_recherche` contient un second dictionnaire qui rassemble le modèle `gpt-4`, la température initiale 0,2, et une liste de deux tuples d'outils et versions : `("WebSearch", "v2.1")` et `("Calculatrice", "v1.0")`. La température est modifiée à 0,5 sans recréer le dictionnaire. L'accès imbriqué :

```
print(config_ia["agent_recherche"]["outils"][1][0])
```

affiche uniquement `Calculatrice`. Cet exercice manipule une configuration locale : aucune API de modèle n'est appelée.

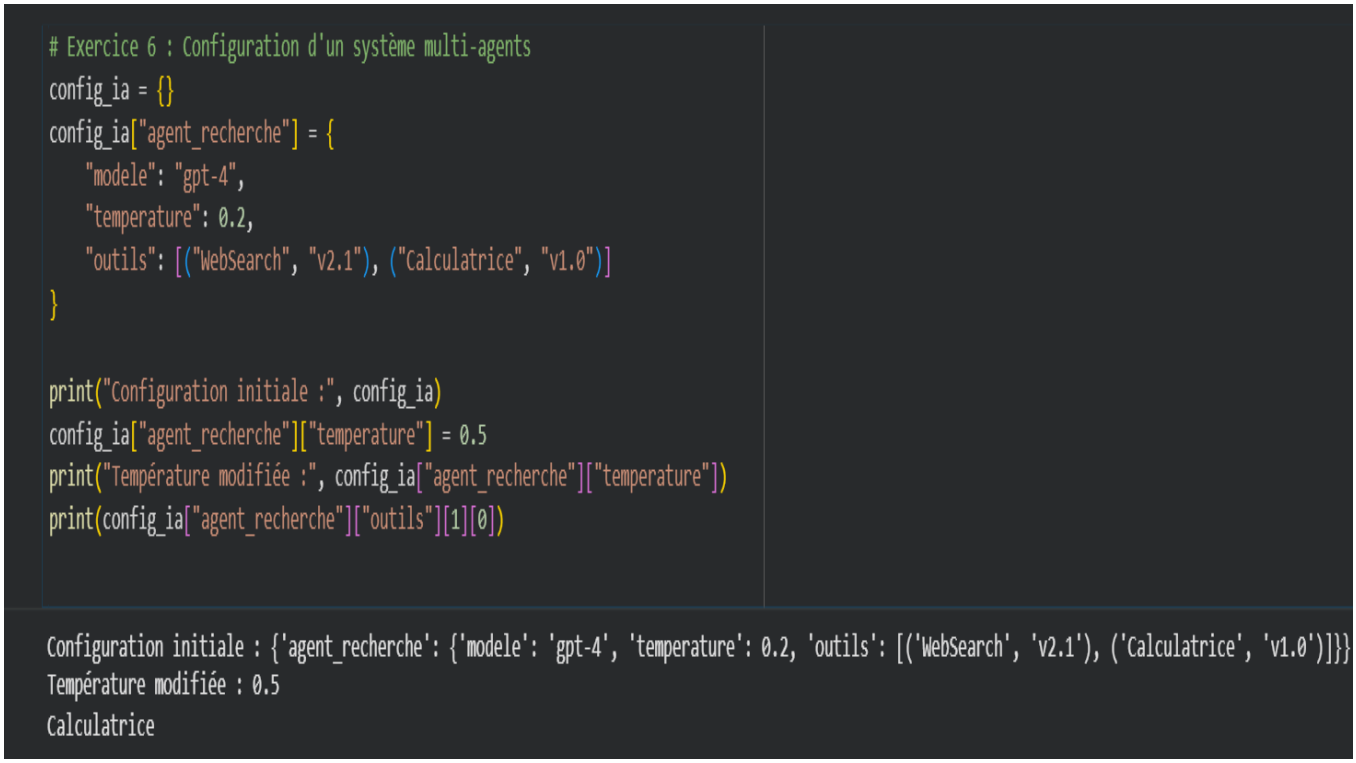


FIGURE 6 – Exercice 6 : configuration, modification et extraction de l'outil Calculatrice

9. Bilan et conclusion

Ces exercices ont permis de travailler les conversions de types, les conditions imbriquées, la répétition avec `while` et `for`, les fonctions, les listes triées ainsi que les dictionnaires et tuples. Le notebook est conçu pour être exécuté dans Google Colab. Les emplacements d'illustration seront remplacés par les véritables captures d'exécution avant la remise.